

Project Progress: Automated DORA Metrics Collection Using OpenTelemetry

Abstract

Large organizations with hundreds of repositories require standardized and highly distributed metric collection solutions to identify risk and oversee portfolio management. The DevOps Research and Assessment (DORA) framework defines four standardized metrics to track across projects:

- **Deployment Frequency:** Release cadence to production
- **Lead Time for Changes:** Duration from commit to production deployment
- **Change Failure Rate:** Percentage of deployments causing production failures
- **Time to Restore Service:** Recovery time from production incidents

However, manual collection of these metrics is labor intensive, error-prone, and lacks standardization. This project implements automated DORA metrics collection and publishing using an OpenTelemetry Collector to scrape GitHub repository data and OpenObserve for storage and visualization.

1 Introduction and Background

1.1 Project Overview

This project automates the collection of software delivery performance metrics from GitHub repositories using the DORA framework established by Forsgren et al. [1]. This system targets organizations seeking to assess their DORA posture through continuous, automated measurement of development practices without manual data extraction or custom CI-CD pipelines.

1.2 System Architecture

The system deploys three components via Docker Compose:

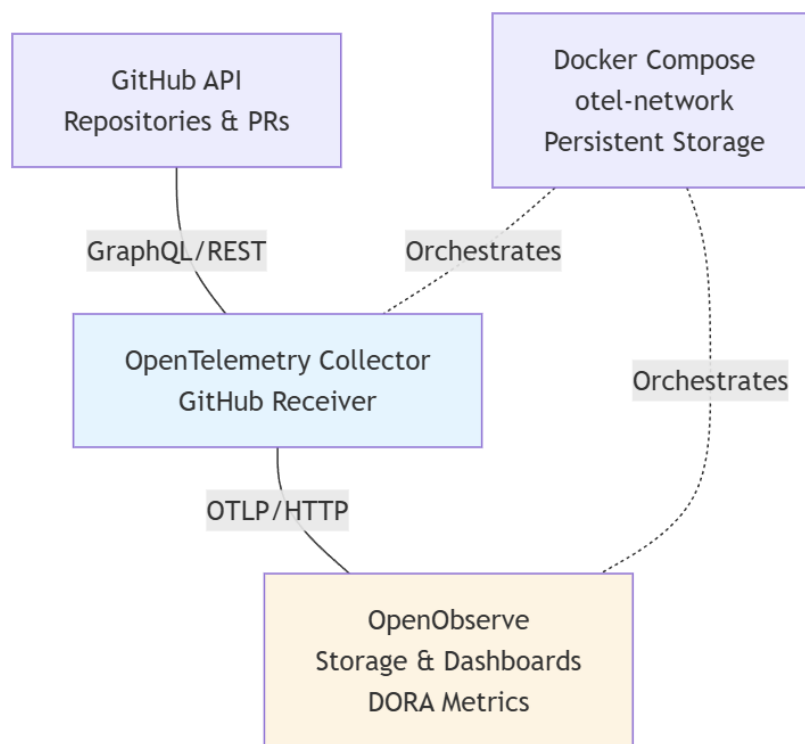


Figure 1: System Architecture Overview

OpenTelemetry Collector (otelcol-contrib) scrapes GitHub repositories every 60 sec-

onds using the GitHub receiver. Authentication uses a developer’s Personal Access Token (PAT) via bearer token extension. The collector exports metrics to OpenObserve via OTLP HTTP protocol.

OpenObserve stores time-series metrics and provides SQL-based querying for dashboard visualization. Deployed on port 5080, it receives OTLP data at `/api/default/` endpoint.

Docker Networking connects both services via bridge network (`otel-network`) with persistent storage (`openobserve-data` volume) for metric retention.

The GitHub receiver supports flexible search queries (e.g., `org:<ORG>`, `is:private`, `is:pr is:open`) enabling targeted collection across repositories and pull request states.

2 Related Work

2.1 DORA Metrics Framework

Forsgren, Humble, and Kim [1] established four metrics that distinguish high-performing software teams based on multi-year research across thousands of organizations:

- **Deployment Frequency:** Release cadence to production
- **Lead Time for Changes:** Duration from commit to production deployment
- **Change Failure Rate:** Percentage of deployments causing production failures
- **Time to Restore Service:** Recovery time from production incidents

Their research demonstrated strong correlations between these metrics and organizational outcomes including profitability, productivity, and market share.

2.2 Automated Metrics Collection

OpenTelemetry [2] provides vendor-neutral observability instrumentation. The GitHub receiver [3], currently at alpha stability, scrapes VCS metrics via GitHub’s GraphQL and REST APIs. Kalliamvakou et al. [4] demonstrated the feasibility of deriving development process metrics from version control systems, though they identified challenges in data completeness and interpretation.

This project leverages the OpenTelemetry GitHub receiver to collect VCS metrics aligned with OpenTelemetry semantic conventions v1.37.0, ensuring standardized telemetry representation across platforms.

3 Preliminary Data

3.1 Collected Metrics

The GitHub receiver collects nine default metrics organized by their corresponding DORA metrics, as shown in Table 1. The GitHub receiver also collects telemetry attributes that provide dimensional and resource context for filtering and aggregating metrics across repositories, organizations, and change states. All metrics conform to OpenTelemetry semantic conventions v1.37.0 for VCS telemetry.

Change failure rate and time to restore service metrics have not yet been captured. Change failure rate requires GitHub Actions workflow status or incident issue correlation. Time to restore service requires access to deployment metrics from a production kubernetes cluster, which may not be feasible for this project.

Metrics Not Yet Captured:

- **Change Failure Rate:**
- **Time to Restore Service:**

Table 1: GitHub Receiver Metrics

Metric Name	Description
<i>Deployment Frequency Support</i>	
<code>vcs.change.count</code>	Pull request counts by state (open/merged)
<code>vcs.repository.count</code>	Total repositories monitored
<i>Lead Time for Changes Support</i>	
<code>vcs.change.time_to_merge</code>	Duration from PR creation to merge (seconds)
<code>vcs.change.time_to_approval</code>	Duration from PR creation to approval (seconds)
<code>vcs.change.duration</code>	Time PRs remain in open state (seconds)
<i>Development Velocity Indicators</i>	
<code>vcs.ref.count</code>	Branch/tag counts per repository
<code>vcs.ref.time</code>	Branch lifespan from creation (seconds)
<code>vcs.ref.lines_delta</code>	Code churn (lines added/removed) vs. trunk
<code>vcs.ref.revisions_delta</code>	Commit count ahead/behind trunk
<i>Telemetry Attributes</i>	
<code>vcs.repository.name</code>	Repository identifier
<code>vcs.repository.url.full</code>	Complete repository URL
<code>vcs.change.state</code>	PR state (open, merged)
<code>vcs.ref.type</code>	Reference type (branch, tag)
<code>vcs.ref.head.name</code>	Branch name
<i>Resource Attributes</i>	
<code>vcs.provider.name</code>	VCS provider (GitHub)
<code>vcs.owner.name</code>	Organization/user account

3.2 Visualization and Analysis

OpenObserve dashboards visualize collected metrics through six panels:

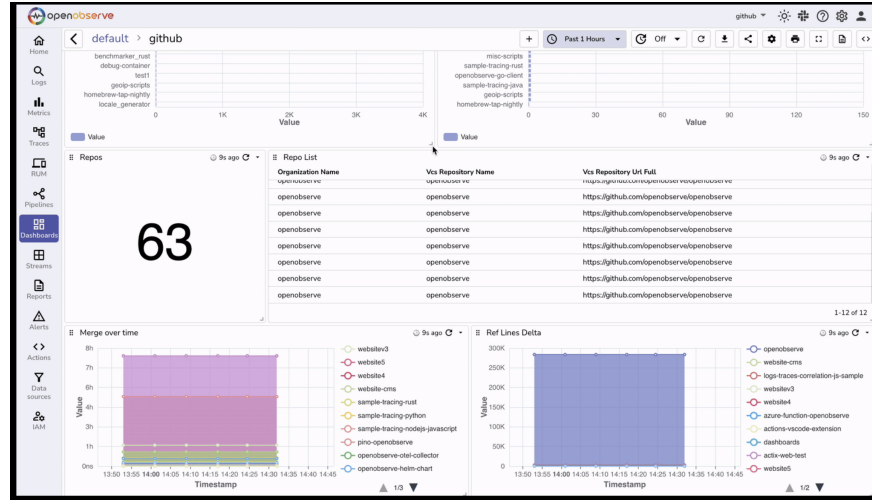


Figure 2: OpenObserve Dashboard Visualizing GitHub Metrics

Repository Activity:

- Repository count (p95 aggregation)
- Top 10 repositories by change volume (horizontal bar chart)
- Reference (branch/tag) count by repository
- Repository list with organization and URL

DORA-Aligned Time Series:

- Merge time trends over time (p95, millisecond precision, repository breakdown)
- Code churn (lines delta) trends over time (repository breakdown)

SQL queries employ 95th percentile aggregation to filter outliers and histogram-based temporal bucketing for trend analysis.

References

- [1] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. IT Revolution Press, 2018.
- [2] OpenTelemetry Authors, “OpenTelemetry specification,” Cloud Native Computing Foundation, 2021, accessed: 2025-04-19. [Online]. Available: <https://opentelemetry.io/docs/specs/otel/>
- [3] OpenTelemetry Collector Contributors, “GitHub receiver,” <https://github.com/open-telemetry/opentelemetry-collector-contrib/blob/main/receiver/githubreceiver/README.md>, 2024, openTelemetry Collector Contrib Repository. Accessed: 2025-04-19.
- [4] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, “The promises and perils of mining GitHub,” in *Proceedings of the 11th Working Conference on Mining Software Repositories*, ser. MSR 2014. Hyderabad, India: ACM, 2014, pp. 92–101.